

Summative Assignment

Module code and title	COMP4231 Data Compression and Coding Theory
Academic year	2025-26
Coursework title	Data Compression assignment
Coursework credits	10 credits
% of module's final mark	50%
Lecturer	Ioannis Ivrissimtzis
Submission date*	Thursday, December 11, 2024 14:00
Estimated hours of work	20 hours
Submission method	Ultra
Additional coursework files	canterburyBenchmark.zip dataCompressionCoursework-Latex.zip dataCompressionCoursework-Word.docx
Required submission items and formats	dcCOMP4231.zip (a zip file containing the Python code, any data files used in your experiments that were not given as part of the assignment, and the pdf report)

* This is the deadline for all submissions except where an approved extension is in place.

Late submissions received within 5 working days of the deadline will be capped at 40%.

Late submissions received later than 5 days after the deadline will receive a mark of 0.

It is your responsibility to check that your submission has uploaded successfully and obtain a submission receipt.

Your work must be carried out by you only and comply with the university rules about plagiarism and collusion. Students suspected of plagiarism, either of published or unpublished sources, including the work of other students and the use of AI tools such as ChatGPT or Gemini, or of collusion, will be dealt with according to University guidelines:

<https://durhamuniversity.sharepoint.com/teams/LTH/SitePages/6.2.4.aspx>

Data Compression and Coding Theory COMP4231

2025/2026

Data Compression assignment

Submit your work on **Ultra** before **11 December 2025, 14:00**. For any questions, contact the setter of the assignment Dr Ioannis Ivrissimtzis: ioannis.ivrissimtzis@durham.ac.uk

Task description

[Describe](#), [implement](#), [test](#), and [analyse](#) the LZW algorithm for data compression.

What to submit

You should submit a single [.zip](#) file named [dcCOMP4231.zip](#) containing:

1. your Python code file(s) together with a [readme.txt](#) file with instructions on how to run your code
2. any test data files mentioned in your report which are not in the Canterbury benchmark
3. your report as a [.pdf](#) file.

Program specification

Your program should be able to compress and decompress, both text and binary files.

The program should allow the user the option to choose between two different data structures for the implementation of the LZW dictionary.

Program testing requirements

Test your program on the Canterbury benchmark.

Test the behaviour of your program on edge cases, such empty files, very small or very large files, and where appropriate other types of files that naïve implementations of the LZW compression algorithm cannot handle.

You can further test your program on other benchmark datasets that you either found online or constructed yourself, as long as they will contribute towards a deeper and more reliable analysis of the algorithm and your implementation.

Structure and contents of the report

You can format your report using the Overleaf Latex template or the Word template uploaded on Ultra, or use any other two-column, single line spacing format, keeping the 2cm page margins.

Your report should not exceed **4 pages**, including figures, tables, references, and appendices. It should be structured in sections, as specified below:

Introduction: Introduce the field of data compression, providing context and motivation for the rest of the report. Summarise your work on the assignment and briefly discuss your main findings.

The LZW algorithm: Give a detailed description of the LZW algorithm; the reader should be able to understand the main algorithm, its main strengths and limitations, and where it can be applied. Briefly describe some of its most important variants and discuss their differences from the original algorithm.

Implementation and testing: Describe your implementation, the main issues encountered and how they were solved. Describe the two data structures you used for implementing the LZW dictionary, justify your choice by a comparison of the advantages and disadvantages of the various possibilities. Discuss your testing strategy, especially regarding the identification of edge cases, and what you did to make your program able to handle them.

Results and analysis: Describe the evaluation metrics used. Report compression ratios on the Canterbury benchmark and, where possible, compare and discuss the performance of LZW against different compression methods. Report results related to the computational and the memory efficiency of your implementation, including the computational and memory requirements of both the compression and decompression processes. The reader should be able to understand how the data structures of the LZW dictionary, and their implementation, affect these requirements, and thus, the limitations of the implementation in terms of the type and size of the files that can handle.

Conclusions: Summarise the main findings of your work. If appropriate, make recommendations for further investigations.

List of references: All the references in the list should be cited somewhere in the main report. You can use any of the reference styles that are commonly used in scientific papers.

Marking procedure

The marking of your submission will mostly be based on the report.

By reading the report one should be able to get a clear idea on what has been implemented and how, and how it was used to investigate the properties of the LZW algorithm.

I will look at the code only to assess its quality, including the quality of the comments. I will run it only to assess its correctness and robustness, without necessarily trying to replicate all your results.

Marking scheme

Program		
a	<u>Correctness</u> The code should be able to compress and decompress the files in the Canterbury benchmark, and the decompressed files should be identical to the original files. The user should have a choice between two different data structures for the implementation of the LZW dictionary, and the program should work correctly for both.	20
b	<u>Robustness</u> The code should be able to handle empty and very small input files, and if necessary, output appropriate warnings when memory bounds are exceeded during either compression or decompression. The code should also be able to handle edge cases of the LZW algorithm that make naïve implementations fail.	5
c	<u>Efficiency</u> At least one of the choices of data structure for the LZW dictionary should lead to an implementation that is efficient in terms of computational power and memory usage during compression and decompression.	5
d	<u>Code quality and comments</u> Good programming practices have been applied as, for example, consistent naming conventions and no code duplication. The comments are informative and help one understand the overall structure of the code, what each part is doing, and the flow of the program.	5
Total		35

Report		
a	<u>Introduction</u> Clear and accurate discussion of the field of data compression, well supported by appropriately cited references. Clear, brief, high-level description of the LZW algorithm, and an informative summary of your work and your main findings.	5
b	<u>The LZW algorithm</u> Clear, accurate, appropriately detailed description of the LZW algorithm and its application domains. Evidence of critical analysis in the discussion of its strengths and limitations. Evidence of independent reading and critical analysis in the description and the comparative discussion of the LZW variants.	10
c	<u>Implementation and testing</u> Clear discussion of the implementation issues encountered and the given solutions, evidence of problem-solving skills. Discussion and comparative analysis of several choices of data structure for the LZW dictionary; the choice of the two data structures is well-motivated and justified. Clear description of an appropriate testing strategy employed, convincing the reader of the correctness and robustness of the implementation.	15
d	<u>Results and analysis</u> Suitable choice of evaluation metrics. All algorithmic performance aspects have been considered; compression ratios, computation power, and memory requirements, during compression and decompression. The results of the tests are presented with clarity, are critically analysed, and were appropriately compared against results found in the literature. The discussion of the effect of the choice of data structure is clear, it is well-supported by the results, and where appropriate explained through the properties of the data structures. Evidence of critical analysis and clarity in the discussion of the limitations; the reader gets a good understanding of what the implementation can handle in terms of type and size of data files.	15
e	<u>Conclusions</u> A concise, clear, concluding summary of the main findings. If appropriate, suitable recommendations for future investigations. The discussion is well-grounded on the previous sections and follows naturally from them; do not use this section for writing some afterthoughts.	5

Report		
f	<u>References</u> The literature has been researched at an appropriate breadth, evidence of independent reading and critical analysis of the findings. The references are of high quality, relevant, and well-integrated in the main text.	10
g	<u>Writing and presentation quality</u> Concepts and ideas are conveyed with clarity. Mature technical language is used, no linguistic mistakes or typos. The paper is well-formatted. If appropriate, use of high-quality tables, diagrams or images.	5
Total		65